Programación Orientada a Objetos

Punteros y Gestión Dinámica de Memoria

Punteros y Gestión Dinámica de Memoria

Introducción

Temas:

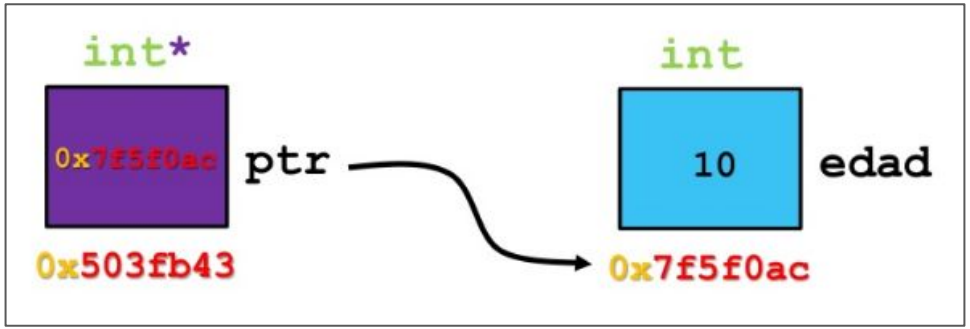
- **Punteros**: Gestión de memoria.
  - Concepto de Puntero
  - Punteros y Arrays
  - Punteros de Cadenas
  - Aritmética de Punteros
  - Punteros como Argumentos de Funciones
  - Punteros a Funciones
  - Punteros a Estructuras
- **Gestión Dinámica de la Memoria**
  - Operadores new y delete
  - Ejercitación y Ejemplos Prácticos

Punteros y Gestión Dinámica de Memoria

Punteros - Concepto

Gestión de Memoria

- Un **puntero** es una variable que almacena una **dirección de memoria**, no un valor directo.
- Se declaran con el operador * (asterisco).
- Se obtiene la dirección de una variable con **&** (operador de dirección).
- Se accede al valor al que apunta con * (operador de desreferenciación o indirección).




```
// Variable entera
int numero = 10;
// Puntero a 'numero'
int* punteroNumero = &numero;
// Imprime 10 (valor al que apunta)
std::cout << *punteroNumero;
```

Punteros y Gestión Dinámica de Memoria

Punteros - Concepto

- Una variable de tipo puntero puede guardar direcciones de variables de un tipo determinado: punteros a int, a double, a char, etc.
- **tipo_de_dato *nombre_variable;**

Ejemplo:

- int i = 3, *p, *r; // p y r son punteros a entero
- double d=3.3, *q; // q es un puntero a double
- char c='a', *t; // t es un puntero a carácter
- p = &i;
- r = p;
- p=q; // ***ERROR*** porque son punteros de diferente tipo

Punteros y Gestión Dinámica de Memoria

Punteros - Operaciones con Punteros

- cin>>*p; // Muestra dato apuntado por p
- if (p==r) // Comparación punteros
- cout<<"El valor de i es: "<<*p;
- *p=*p+3; // Suma 3 unidades al dato apuntado por p
- p=p+3; // Desplaza el puntero 3 posiciones

La magnitud del desplazamiento está dada en función del tipo de dato al que apunta el puntero.

Es posible desplegar una dirección como valor de una variable de tipo puntero, o la dirección de cualquier variable, utilizando cout.

Ejemplo:

```
int n = 5; *p;

cout << “p = “ << p << endl;

cout << “&n = “ << &n << endl;

cout << “&p= “ << &p << endl;
```

Punteros y Gestión Dinámica de Memoria

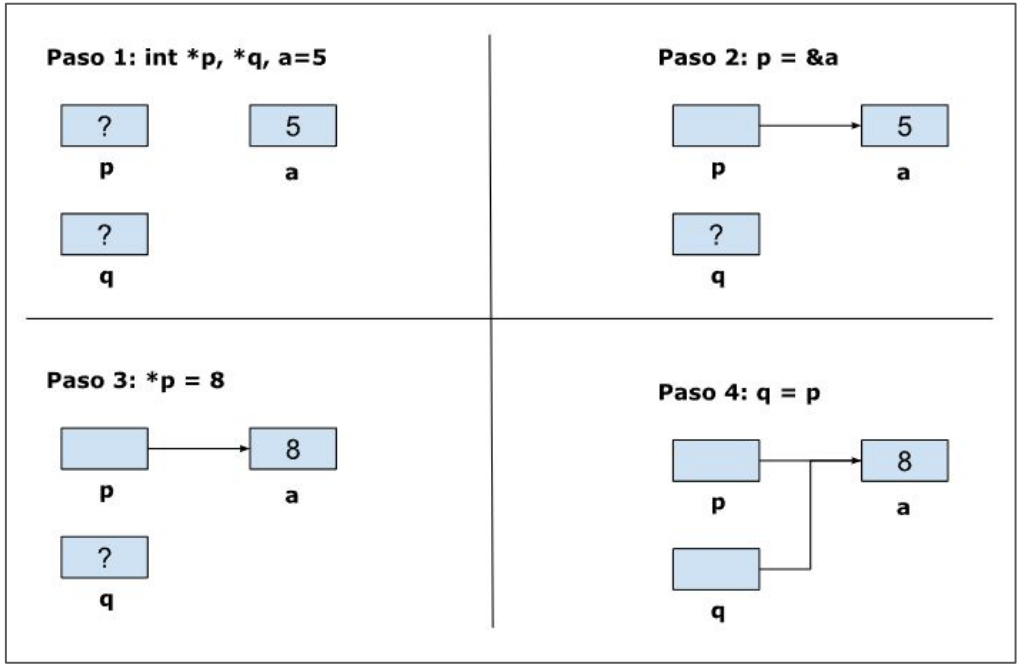
Punteros - Seguimiento

Secuencia de Programa

…

```
int *p, *q, a = 5;
p = &a;
*p = 8;
q = p;
*q = 85;
a = 33;
int b = 1;
q = &b;
```

…



Punteros y Gestión Dinámica de Memoria

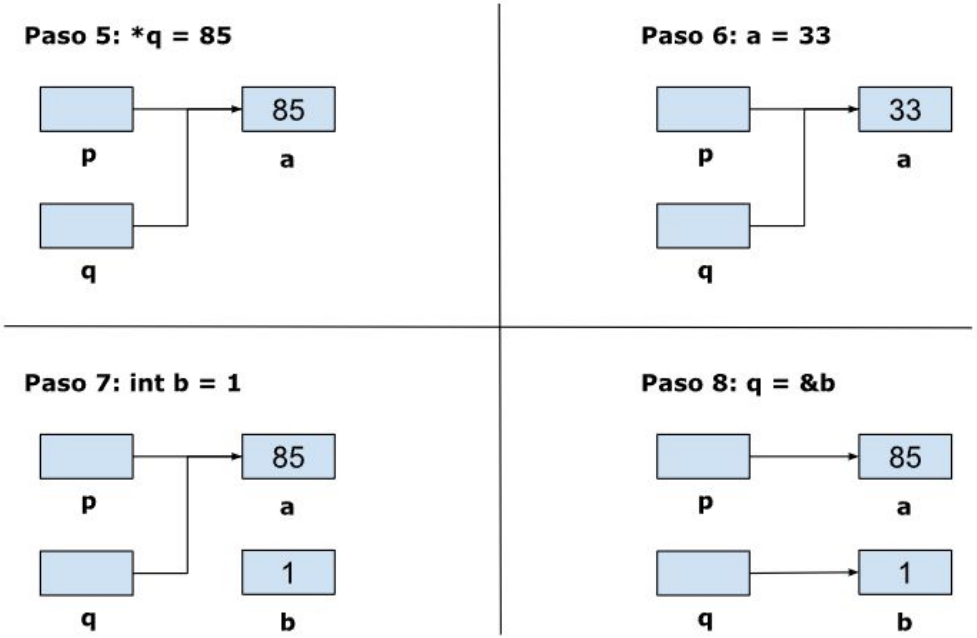
Punteros - Seguimiento

Secuencia de Programa

…

```
int *p, *q, a = 5;
p = &a;
*p = 8;
q = p;
*q = 85;
a = 33;
int b = 1;
q = &b;
```

…



Punteros y Gestión Dinámica de Memoria

Punteros y Arrays

- El nombre de un **array**, sin índice, se comporta como un **puntero al primer elemento**.
- Esto permite recorrer arrays de forma eficiente.

```
// Declaración de arreglo de enteros
int miArray[] = {10, 20, 30};

// Puntero 'p' apunta a miArray[0]
int* p = miArray;
std::cout << *p;  // Imprime 10
// Imprime 20 (apunta a miArray[1])
std::cout << *(p + 1);
// Imprime 30 (apunta a miArray[2])
std::cout << *(p + 2);
```

Punteros y Gestión Dinámica de Memoria

Punteros de Cadenas

- En C++, las cadenas de caracteres son **arrays de char** terminados en \0 (carácter nulo).
- Los punteros son ideales para manipularlas.

```
// Define cadena de texto
char mensaje[] = "Hola";

// 'ptrMensaje' apunta al primer carácter
char* ptrMensaje = mensaje;

// Recorriendo la cadena con el puntero
while (*ptrMensaje != '\0') {
    std::cout << *ptrMensaje;
    ptrMensaje++; // Avanza al siguiente carácter
}

// Resultado: Hola
```

Punteros y Gestión Dinámica de Memoria

Aritmética de Punteros

- Podemos sumar o restar enteros a un puntero.
- El puntero se mueve en "saltos" del **tamaño del tipo de dato** al que apunta.

```
// Declaración de arreglo
int numeros[] = {10, 20, 30, 40};
// Puntero 'p' apunta a 10 (Primer dato del
arreglo)
int* p = numeros;
// 'p' ahora apunta a 20 (se mueve sizeof(int)
bytes)
p = p + 1;
std::cout << *p; // Imprime 20
p = p + 2;       // 'p' ahora apunta a 40
std::cout << *p; // Imprime 40
```

Punteros y Gestión Dinámica de Memoria

Punteros como Argumentos de Funciones: Paso por referencia

- Permite que una función **modifique el valor de una variable** fuera de su ámbito local.
- Conocido como "paso por referencia".

```
// Función duplicar del programa
void duplicar(int* valor) {
     // Modifica el valor original
    *valor = *valor * 2;
}

int main() {
    int miNumero = 5;
    // Pasamos la dirección de miNumero
    duplicar(&miNumero);
    std::cout << miNumero; // Imprime 10
    return 0;
}
```

Punteros y Gestión Dinámica de Memoria

Punteros a Funciones: Flexibilidad en el código

- Un puntero puede **apuntar a una función**.
- Útil para **callbacks**, tablas de dispatch, y algoritmos genéricos.

```
int sumar(int a, int b) { return a + b; }
int restar(int a, int b) { return a - b; }

int main() {
    // Declara un puntero a una función que toma dos int
y devuelve un int
    int (*ptrOperacion)(int, int);
    // Apuntamos a Función sumar
    ptrOperacion = sumar;
    std::cout << ptrOperacion(5, 3); // Imprime 8
    // Apuntamos a Función restar
    ptrOperacion = restar;
    std::cout << ptrOperacion(5, 3); // Imprime 2
    return 0;
}
```

Punteros y Gestión Dinámica de Memoria

Punteros a Estructuras (y Clases): Accediendo a objetos

- Muy usados para acceder a los **miembros de estructuras/clases** y para pasar objetos eficientemente.
- Se usa el operador **->** (flecha) para acceder a los miembros a través del puntero.

```
// Definimos un tipo de dato “compuesto”
struct Persona {
    std::string nombre;
    int edad;
};

int main() {
     // persona1 es una variable del tipo Persona
    Persona persona1 = {"Ana", 30};
    Persona* ptrPersona = &persona1;

    // Acceso a miembros usando el operador flecha (->)
    std::cout << ptrPersona->nombre; // Imprime Ana
    std::cout << ptrPersona->edad;   // Imprime 30
    return 0;
}
```

Punteros y Gestión Dinámica de Memoria

Gestión Dinámica de la Memoria: Control total

Características

- Permite asignar memoria en el **heap** (almacenamiento libre) durante la **ejecución** del programa.
- La memoria en el heap no está limitada por el ámbito de las funciones.
- Es crucial para estructuras de datos dinámicas (listas, árboles) o cuando el tamaño de los datos es variable.

Ventajas

- **Flexibilidad**: Adaptarse a necesidades de memoria en tiempo real.
- **Eficiencia**: Evitar el consumo excesivo de memoria para variables no siempre necesarias.

Punteros y Gestión Dinámica de Memoria

Operadores new y delete

- **new**: Asigna memoria dinámicamente.
- Devuelve un puntero al bloque de memoria asignado.
- Si falla, lanza std::bad_alloc. (Error)

```
// Asigna memoria para un entero
int* ptrEntero = new int;
*ptrEntero = 100;

// Asigna memoria para un array de 5
enteros
int* ptrArray = new int[5];
ptrArray[0] = 10;
```

- **delete**: Libera la memoria asignada con new.
- Evita las **fugas de memoria (memory leaks)**.
- delete para un solo objeto, delete[] para arrays.

```
// Libera memoria de un solo entero
delete ptrEntero;
// Buena práctica
ptrEntero = nullptr;
// Libera memoria de un array
delete[] ptrArray;
ptrArray = nullptr;
```

Punteros y Gestión Dinámica de Memoria

Conclusiones

- Los **punteros** son herramientas poderosas para manipular memoria directamente.
- La **gestión dinámica de memoria** con new y delete permite flexibilidad y eficiencia.
- **Importante**: Toda memoria asignada con new debe ser liberada con delete (o delete[]).
- Dominar estos conceptos es esencial para escribir código robusto y sin fugas de memoria en POO.